

CW1.1: Part 1 Take-home Assignment

Preamble

Mode: **Group** assessment. Students work in groups of five students; groups are allocated directly by the teaching team.

Type: Formative assessment, prerequisite for In-Class Test 1 (CW1.2).

Deadline: **Monday 13th April (Week 7), 09:00 (China Standard Time).**

Feedback: General feedback will be provided to the entire cohort during the Week 7 lectures. Individual groups may request specific feedback via email and through in-person group consultation sessions (20 minutes per group, bookable through Learning Mall).

Description: You will solve the *Optimal Synchronised Schedule* problem described in the Overview document, using a brute-force approach and array-based data structures. Detailed instructions are provided below.

Submission: Your group is required to submit the following documents to the dedicated “CW Phase 1 - Take-home Assignment” submission activity on Learning Mall:

- A **Java source code solution file**, named *Phase1_Group{X}_Soln.java*.
- A concise group report of no more than 6 pages (Appendix excluded), as a PDF file named *Phase1_Group{X}_Report.pdf*, and containing the following sections:
 - High-level overview of your solution, including explanation of data structures used and algorithms developed;
 - An analysis of the efficiency of your solution, applying notions of complexity and Big-Oh estimates from the lectures
 - A reflection on the limitations of your solution, and a discussion of possible future improvements.
 - Where applicable, an appendix detailing AI usage, as explained in the *AI Usage Statement* below (not included in page limits).
- A test folder, compressed into a ZIP archive named *Phase1_Group{X}_Test.zip*, containing both supplied test maps and additional maps of your own design; this should include at least one impossible case (where no synchronised schedule exists).

Note that group submissions settings have been enabled on Learning Mall, so that only one submission per group is required.

AI Usage Statement: Generative artificial intelligence (GenAI) is a rapidly evolving technology with implications for graduate study. It is important to note that the **University's academic integrity policy has not changed**. For the take-home part of this project, you are permitted to make **substantive use of AI**. You may employ AI tools to assist you in **any part of solving the problem**, including but not limited to brainstorming solution ideas, generating Java code, and assistance with report-writing. However, the summative assessment for this coursework will be an on-site in-class test based on your take-home work, during which the use of AI tools will be strictly prohibited. As such, you must ensure you have a complete understanding of your solution, irrespective of any AI usage in developing it. In the appendix of your report, you should provide a full disclosure of your AI usage, including the tool name, version, and access date, and a detailed description of the AI's role, e.g.:

ChatGPT-5, accessed 10th April, 2026, generated Java code for *<insert functionality here>* based on a description of the task in natural language.

Late Submission: As this is a formative assignment, there will be no penalty for late submission. However, access to the summative in-class test will be limited to students whose group has a valid submission for this assignment on Learning Mall.

Academic Integrity Statement: This assignment is group work. Please refer to the Overview document's statement on *Collaboration and Academic Integrity*. Plagiarism (e.g., copying materials from other sources without proper acknowledgement) is a serious academic offence. Plagiarism, collusion, copying, unauthorised assistance, dishonest use of data, and academic offences involving AI will not be tolerated and will be dealt with in accordance with the University Academic Integrity Policy. Submitting work created by others, whether paid for or not, is a serious offence, and will be prosecuted vigorously. The use of AI tools beyond the scope outlined above would also be considered in breach of the University Academic Integrity Policy, and dealt with accordingly.

Detailed solution instructions

The remainder of this document is dedicated to providing you with some detailed guidance on how to solve your assigned task (i.e., the coding aspects). Note that this is intended for guidance only. Variations on the different steps and functions are possible, within the constraint that your solution should only make use of array and list-based data structures.

1. Map encoding

Hospital map examples are provided as plain-text rectangular grids. More precisely, each hospital map is an array of Strings, where each String represents a row of the map. All rows have equal length. The following symbols are used.

#	Wall / blocked cell
.	Free cell; there are P free cells in total
A	Starting position S_A of Robot A
B	Starting position S_B of Robot B
a	Target T_A of Robot A
b	Target T_B of Robot B

2. Free cell indexing

You should begin by indexing the free cells in the map by 0, ..., P-1 using arrays. While the actual indexing order is unimportant, the following information should be easily retrievable:

- Given an index i in 0, ..., P-1, retrieve the row and column numbers (in the hospital grid map) of the free cell indexed by i .
- Given a free cell in row r and column c in the hospital grid map, retrieve its index i in the free cell indexing.

We refer to the index of a free cell in 0, ..., P-1 as its *identifier*. Figure 1 shows an example of a hospital grid map. On the left is the array of Strings encoding the map. On the right is a grid representation of the map, with free cells indexed in so-called lexicographical order. Rows and columns of the grid are numbered from top to bottom and left to right, with numbering starting at 0. For example, the free cell in row 1, column 4 has identifier x, while the cell with identifier y has row number 3 and column number 2. To ease notation, the map has been left empty, without robots or targets. The cells containing the robots' starting positions and targets are free cells, so should be included in the initial indexing described above.

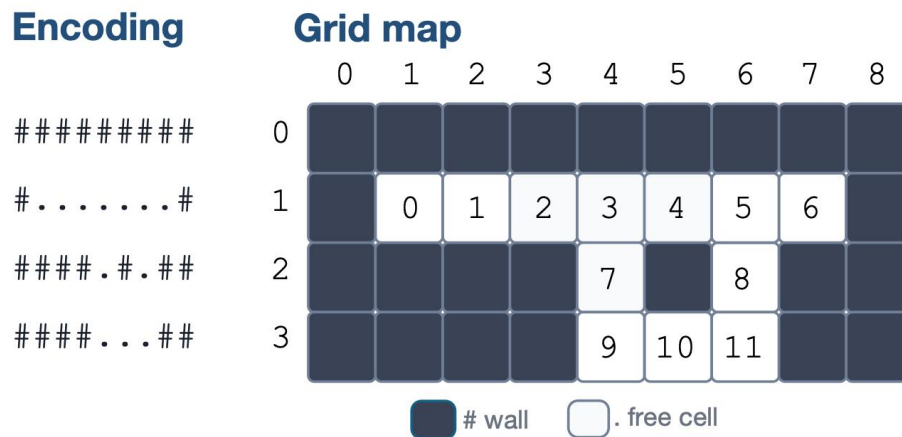


Figure 1: Indexing of free cells in a hospital map

3. Move computation

You should write a procedure that, given a free cell $\underline{c}$ in the map, generates the list of free cells that can be reached through a single move from $\underline{c}$. One possibility is to write a Java method `computeSuccessors(int identifier)` that returns the list of identifiers of free cells that can be reached from the cell indexed by the given `identifier`. For example, in the map on Figure 1, `computeSuccessors(3)` would return the list `[2, 3, 4, 7]`, where the elements identify cells that can be reached through the moves W, WAIT, E, S in that order. Note that the ordering of the list produced is unimportant: the key is to reject any move which would hit a wall (in this example, the move N should be rejected).

4. The key step: computing reachable states

The key step in solving the problem will be to compute the set of *reachable states* at time t , for $t = 0, 1, \dots, T$, where T denotes some fixed maximal search time. Formally, a *state* is a pair (a, b) of free cell identifiers, indicating that Robot A occupies the cell with identifier a , while Robot B occupies the cell with identifier b . The set of reachable states at times $t \geq 0$ is defined recursively as follows:

- At time $t = 0$, the only reachable state is (S_A, S_B) , the identifiers of the starting positions of the two robots.
- For $t \geq 1$, a state (a, b) is reachable at time t if there exists a state (a', b') which is reachable at time $t-1$ such that the pair of moves $(a \rightarrow a', b \rightarrow b')$ is legal. Recall from the Overview document that a pair of moves is legal if the robots do not swap positions or end up in the same cell. We will say that the state (a', b') is *reachable from* (a, b) in such cases.

For example, consider the map in Figure 1, and the state (2, 3) indicating that Robot A occupies the cell with identifier 2 and Robot B the cell with identifier 3. Then the set of legal move pairs from this state, and the corresponding states reached, are as follows:

- Move (W, W), reaching the state (1, 2);
- Move (W, E), reaching the state (1, 4);
- Move (W, S), reaching the state (1, 7);
- Move (W, WAIT), reaching the state (1, 3);
- Move (E, E), reaching the state (3, 4);
- Move (E, S), reaching the state (3, 7);
- Move (WAIT, E), reaching the state (2, 4);
- Move (WAIT, S), reaching the state (2, 7);
- Move (WAIT, WAIT), reaching the state (2, 3).

Examples of illegal moves include (WAIT, W), which would result in both robots occupying the cell with identifier 2, or (E, W), which would result in the two robots swapping cells.

A possible approach to computing reachable states is to employ a 3-dimensional array of booleans `reachable[t][a][b]`, with ranges $0 \leq t \leq T$, $0 \leq a < P$, $0 \leq b < P$. In this array, `reachable[t][a][b]` is `true` if the state (a, b) is reachable at time t, and `false` otherwise. The array can be computed iteratively (over the parameter t) by computing the set of reachable states at time t from those reachable at time t-1, as described above.

5. Task description

Your program should provide a `solve(String[] map, int T)` method that takes as parameters a map encoding as described in Section 1 above, and a maximal search time T. It should determine whether there exists a synchronised schedule for the input map with makespan at most T via the strategy described in Sections 2 to 4 above, and return the makespan of an optimal synchronised schedule, i.e. the minimal value of $t \leq T$ such that `reachable[t][TA][TB]` is true. If no such schedule exists, the method should return -1.

Examples of input maps together with one possible optimal synchronised schedule are provided in the appendix at the end of this document. As well as the examples provided, you should devise your own test maps, including at least one impossible case. All test cases considered should be included in a separate test folder in your Learning Mall submission, as detailed in the **Submission** instructions in the preamble above.

6. Optional task

As well as providing the optimal makespan, it would be helpful if the corresponding synchronised schedule could also be provided. To do so, you should modify the `solve()` method above so that it calculates and returns the optimal makespan and prints one optimal synchronised schedule. The schedule may be printed one tick per line, for example:

tick 1: A=E, B=STAY.

But any equivalent unambiguous notation is acceptable. In order to accomplish this task, you will need to “remember” not just which states are reachable at time t , but also how they can be reached.

This additional task is optional, intended for student groups who wish to attempt a more challenging task. Completing it will lead you to consider more advanced concepts such as back-tracking in dynamical programming. The In-Class Test assessment will not include any questions on this optional task.

7. Algorithm Analysis and Discussion

You should analyse the complexity of your `solve(map, T)` method using the Big-Oh notation seen in the lectures. The algorithm takes two inputs, so should consider the parameters P (number of free cells) and T (maximal search time) in its analysis. Your analysis should also consider any helper methods that your `solve()` method calls, such as the `computeSuccessors()` method detailed in Section 3 above.

Your algorithm analysis should be included in your summary report (see **Submission** details in the preamble of this task sheet). You should then discuss the limitations of techniques employed. For example, you may wish to explain why a bound needs to be placed on the maximal search time, and what happens to your algorithms’ performance when the bound is allowed to increase substantially. You may also discuss possible ways in which your solution could be optimised. For example, are there any legal move pairs which can be automatically discarded when searching for an optimal solution?

Appendix: Representative map examples

The Appendix contains a number of map examples and their expected solution outputs. To simplify notation, in the maps we denote A/B (uppercase) the starting positions of the two robots, and a/b (lowercase) their targets.

1. Single-lane corridor swap

Encoding <pre>##### #bA...Ba# #####</pre>	Illustrative map Legend: # wall . free A/B starts a/b targets
---	--

In this example, no synchronised schedule exists, since the robots would need to pass each other in the corridor, implying a position swap. The `solve()` method should therefore return -1 in this case.

2. Four-way intersection

Encoding <pre>##### ###a### ###.### #A...B# ###.### ###b### #####</pre>	Illustrative map Legend: # wall . free A/B starts a/b targets
---	--

In this example, one of the robots must give way at the intersection by waiting. One possible optimal synchronised schedule is: (E, W), (E, WAIT), (N, W), (N, S), (WAIT, S), which gives an optimal makespan of 5.

3. Parking bay

Encoding <pre>##### #bA...Ba# ####.#### #####</pre>	Illustrative map Legend: # wall . free A/B starts a/b targets
---	--

In this example, a single side bay creates just enough temporary storage for one robot to pull over and allow the other robot past. One possible optimal synchronised schedule is: (E, W), (E, WAIT), (S, W), (N, W), (E, W), (E, W), (E, WAIT), giving an optimal makespan of 7.

4. Side-room staging

Encoding <pre>##### #A....b.# #...### #..B.a...# #####</pre>	Illustrative map Legend: # wall . free A/B starts a/b targets
--	--

In this example, a small chamber supports staging: Robot A detours so that Robot B can finish first. One possible optimal synchronised schedule is: (S, E), (S, E), (E, N), (E, N), (E, E), (E, WAIT), giving an optimal makespan of 6.

5. Sealed target chamber

Encoding

```
#####  
#A..#.a.#  
#.#####  
#B...b#.#  
#####
```

Illustrative map

Legend: # wall . free A/B starts a/b targets

This example presents a case which is impossible because of the basic configuration of the map: there is no way for Robot A to access its target cell a. The `solve()` method should therefore return -1 in this case.

CW1.2: In-class Test 1

Mode: Individual assessment

Weighting: 10% of the overall module mark

Deadline: Week 7 (exact time and date TBC).

Description: The In-class Test 1 (ICT1) is an individual assessment designed to evaluate your understanding of your group's submission in the Part 1 Take-home Assignment (CW1.1), and your knowledge of the first six weeks of the module. Questions will refer specifically to aspects of the CW1.1 task, and access to ICT1 will be restricted to students who have a valid submission for CW1.1 on Learning Mall.

Schedule: During Week 7 (exact time and date TBC).

Content coverage: The test will assess understanding of groups' submissions to CW1.1, together with material covered in Weeks 1–6 of the module.

Format: Six short-answer questions, where the answer is a word, a symbol, an asymptotic form, or a one-line statement; Two analytical questions, where students critically evaluate an algorithm's performance.

- ✧ There will be separate question banks for each type of question, with each student being allocated a random sample from the bank.
- ✧ The test will be set as a Learning Mall Quiz, run on university computers with Safe-Exam Browser enabled to prevent students' use of unauthorised materials.
- ✧ This is a partially open-book examination: students will be allowed access to any PDF electronic materials they bring with them to the examination room.

Late Submission: There is only one opportunity to take this test, **no** late submissions are possible.

Mark Type: Individual mark

Assessment Element	Marks
Questions 1–6 (short-answer)	10 marks per question
Questions 7-8 (analytical)	20 marks per question